本科生计算复杂性理论报告

题　目：　第一次编程作业

组长学号	**23336150**
组长姓名	刘昊
组员**1**学号	**23336203**
组员**1**姓名	任意嵘
组员**2**学号	**23336193**
组员**2**姓名	彭怡萱
专业名称	计算机科学与技术
任课教师	冯剑琳
实验时间	**2026**年**3**月**23**日—**4**月**3**日

目　录

一、实验目的与要求 …… 1

二、模块设计思路 …… 1

三、关键代码展示 …… 2

四、运行结果展示 …… 5

五、总结 …… 9

一、 实验目的与要求

本次编程作业要求以小组为单位，编写 Python 程序实现图灵机编码、多带图灵机以及通用图灵机，并以两数加法图灵机 ADD 为测试用例进行测试。

项目已完成所有模块的实现，具体要求如下：

1) **图灵机编码模块**：读取 JSON 文件并转换为二进制编码，将状态、符号、方向映射为正整数后用一进制表示。

2) **多带图灵机模块**：实现多带图灵机模拟器，支持交互式和自动两种展示模式，实时打印磁带状态和磁头位置。

3) **通用图灵机模块**：基于多带图灵机实现 UTM，使用三条磁带模拟任意单带图灵机，打印运行阶段说明。

4) **用例测试**：构造 ADD 图灵机的 JSON 定义文件，执行 UTM 运转测试。

二、 模块设计思路

(1) 图灵机编码模块

编码模块将图灵机的 JSON 定义转换为二进制字符串。设计过程分为以下步骤：

步骤1：JSON 解析与归一化

读取 JSON 后遍历每个状态及其转移条件，校验核心字段是否存在。同时对符号和方向进行归一化处理，例如将空白符的多种写法统一为单空格，将方向统一为 `left`、`right`、`stay`。

步骤2：状态 ID 映射

采用两阶段映射策略避免 ID 冲突：首先用正则表达式提取符合 `qN` 格式的状态编号；然后对非标准状态按字典序分配剩余的最小正整数。

步骤3：符号与方向 ID 分配

按照规范固定分配：`0`→1，`1`→2，空格→3，其余符号从4开始递增。方向映射为：`left`→1，`right`→2，`stay`→3。

步骤4：转移排序与编码

为保证编码确定性，对转移规则按五维键值排序后，将每个转移编码为一进制串，转移内部用 `1` 分隔，转移之间用 `11` 连接。

(2) 多带图灵机模块

多带图灵机模块实现了一个通用的模拟器，设计过程如下：

步骤1：磁带模型设计

每条磁带用字符列表表示，支持双向无限延伸。磁头位置用整数索引标记，读写操作自动处理边界扩展。

步骤2：转移规则定义

转移规则基于当前状态和所有磁带的读取符号，确定写入符号、移动方向和下一状态。使用元组作为字典键实现高效查找。

步骤3：执行流程实现

每步执行时：读取所有磁头符号 → 查找匹配转移 → 执行写入和移动 → 更新状态。支持交互式（等待用户输入）和自动模式（固定时间间隔）。

(3) 通用图灵机模块

UTM 模块基于多带图灵机实现，按照 `tm2.ppt` 的定义设计：

步骤1：三带架构设计

- **Tape 1**：存储被模拟图灵机的编码 M 和输入 w，格式为 $M111w$
- **Tape 2**：模拟被模拟图灵机的磁带，符号用一进制编码表示
- **Tape 3**：存储被模拟图灵机的当前状态（状态ID的一进制）

步骤2：编码解析

从 Tape 1 解析转移规则：用 `11` 分隔各转移，用 `1` 分隔转移内部各部分，读取连续 `0` 的数量得到数值。

步骤3：执行阶段实现

按照标准步骤执行：Step 2 分析符号表示长度 → Step 3 初始化磁带 → 主循环查找并执行转移 → 判断接受或拒绝。

三、 关键代码展示

(1) 图灵机编码模块

① 转移规则排序

为保证编码结果的确定性，对转移规则按五维键值进行排序：

```
ordered_transitions = sorted(
    transitions,
    key=lambda transition: (
        state_ids[transition.current_state],
        symbol_ids[transition.read_symbol],
        state_ids[transition.next_state],
        symbol_ids[transition.write_symbol],
        DIRECTION_IDS[transition.move],
    ),
)
```

排序后，无论 JSON 中状态的书写顺序如何，同一图灵机始终产生相同的编码字符串。

② 状态 **ID** 映射

采用两阶段策略避免 ID 冲突：

```
def build_state_ids(transitions):
    # 阶段1：提取 qN 格式的状态编号
    for state_name in sorted(state_names):
        match = STATE_NAME_PATTERN.fullmatch(state_name)
        if match:
```

```
            state_ids[state_name] = int(match.group(1))

    # 阶段2：为非标准状态分配剩余ID
    for state_name in sorted(state_names):
        if state_name not in state_ids:
            state_ids[state_name] = next_available
            next_available += 1
```

③ 一进制编码

将转移规则转换为一进制字符串：

```
def encode_transition(transition, state_ids, symbol_ids):
    parts = [
        unary_encode(state_ids[transition.current_state]),
        unary_encode(symbol_ids[transition.read_symbol]),
        unary_encode(state_ids[transition.next_state]),
        unary_encode(symbol_ids[transition.write_symbol]),
        unary_encode(DIRECTION_IDS[transition.move]),
    ]
    return "1".join(parts)  # 转移内部用1分隔
```

最终将所有转移用 11 连接：`"11".join(encodings)`。

(2) 多带图灵机模块

① 磁带类实现

磁带类封装了读写和移动操作：

```
class Tape:
    def __init__(self):
        self.content = ["␣"]  # 初始为空符号
        self.head_position = 0

    def read(self) -> str:
        return self.content[self.head_position]

    def write(self, symbol: str):
        self.content[self.head_position] = symbol

    def move(self, direction: str):
        if direction == "left":
            self.head_position -= 1
        elif direction == "right":
            self.head_position += 1
```

磁带自动处理边界扩展，支持无限延伸。

② 单步执行

核心的 step() 方法实现单步转移：

```python
def step(self) -> bool:
    if self.halted:
        return False

    # 读取所有磁带的当前符号
    current_symbols = tuple(tape.read() for tape in self.tapes)
    key = (self.current_state, current_symbols)

    # 查找匹配的转移规则
    if key not in self.transitions:
        self.halted = True
        return False

    # 执行转移
    transition = self.transitions[key]
    for i, tape in enumerate(self.tapes):
        tape.write(transition.write_symbols[i])
        tape.move(transition.moves[i])

    self.current_state = transition.next_state
    return True
```

(3) 通用图灵机模块

① 磁带初始化

UTM 初始化时设置三条磁带的内容：

```python
def initialize_tapes(self):
    # Tape1: M111w
    tape1_content = self.machine_encoding + "111" + self.input_string
    self.tapes[0].content = list(tape1_content)

    # Tape2: 输入w的符号编码
    tape2_content = self._encode_input_for_tape2(self.input_string)
    self.tapes[1].content = list(tape2_content)

    # Tape3: 起始状态编码
    start_encoding = self._unary_encode(self.simulated_state_id)
    self.tapes[2].content = list(start_encoding)
```

② 转移解析

从编码字符串解析转移规则：

```python
def _parse_transitions(self, encoding: str):
    transitions = []
    for part in encoding.split("11"):  # 转移间用11分隔
        components = part.split("1")    # 转移内部用1分隔
        if len(components) == 5:
            transition = UTMTransition(
                current_state_id=len(components[0]),  # 0的数量
                read_symbol_id=len(components[1]),
                next_state_id=len(components[2]),
                write_symbol_id=len(components[3]),
                move_id=len(components[4])
            )
            transitions.append(transition)
    return transitions
```

③ 主循环执行

UTM 的主循环实现：

```python
def step(self) -> bool:
    if self.state.halted:
        return False

    # 解码当前符号
    current_symbol_id = self._decode_symbol_from_tape2()

    # 搜索匹配转移
    self.state.stage = "search_move"
    transition = self._find_matching_transition(
        self.simulated_state_id, current_symbol_id)

    if transition is None:
        self.state.stage = "no_match"
        self.state.halted = True
        return False

    # 执行转移
    self.state.stage = "execute_move"
    self._execute_transition(transition)

    return True
```

四、 运行结果展示

(1) 图灵机编码模块测试

运行程序选择编码功能，对 ADD 图灵机进行编码：

```
$ python main.py encode --sample add_tm.json --details
```

编码结果输出状态映射、符号映射和转移规则列表：

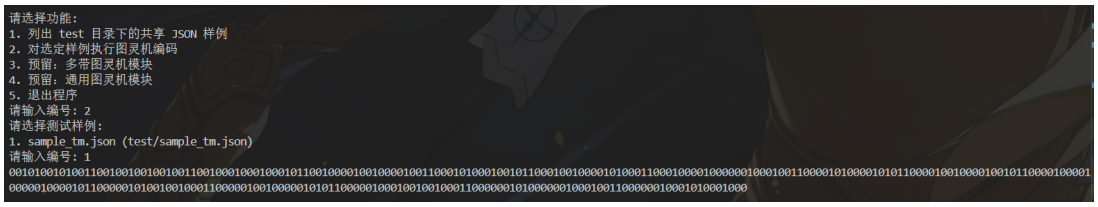


图1 ADD图灵机编码结果

编码成功生成 21 个转移规则，状态映射为 q1→1, q2→2, ..., halt→8，符号映射为 0→1, 1→2, 空格→3。

(2) 多带图灵机模块测试

使用两带图灵机示例进行测试：

```
$ python main.py multitape --mode auto --interval 0.1
```

运行过程展示每步的状态变化：

```
============================================================
多带图灵机模拟器
============================================================
初始状态
当前状态: q1     步数: 0
Tape 1:       *0101
Tape 2:       *

第 1 步
当前状态: q1     步数: 1
Tape 1:       0*101
Tape 2:       *

...

第 7 步
当前状态: halt     步数: 7
Tape 1:       0*110
Tape 2:       *

结果: 接受 (ACCEPTED)
总步数: 7
============================================================
```

```
Tape 2:      *

按 Enter 键继续下一步...
==================================================
第 6 步
==================================================
按 Enter 键继续下一步...
==================================================
第 6 步
==================================================
当前状态: q2
步数: 6
Tape 1:      01*00
----------------------------------------
Tape 2:      *

按 Enter 键继续下一步...
当前状态: q2
步数: 6
Tape 1:      01*00
----------------------------------------
Tape 2:      *

按 Enter 键继续下一步...
Tape 2:      *

按 Enter 键继续下一步...
==================================================
==================================================
第 7 步
==================================================
当前状态: halt
步数: 7
Tape 1:      0*110
----------------------------------------
Tape 2:      *


==================================================
结果: 接受 (ACCEPTED)
==================================================
结果: 接受 (ACCEPTED)
总步数: 7
==================================================
```

图2 多带图灵机运行过程（部分截图）

(3) 通用图灵机模块测试

使用 ADD 图灵机测试 UTM：

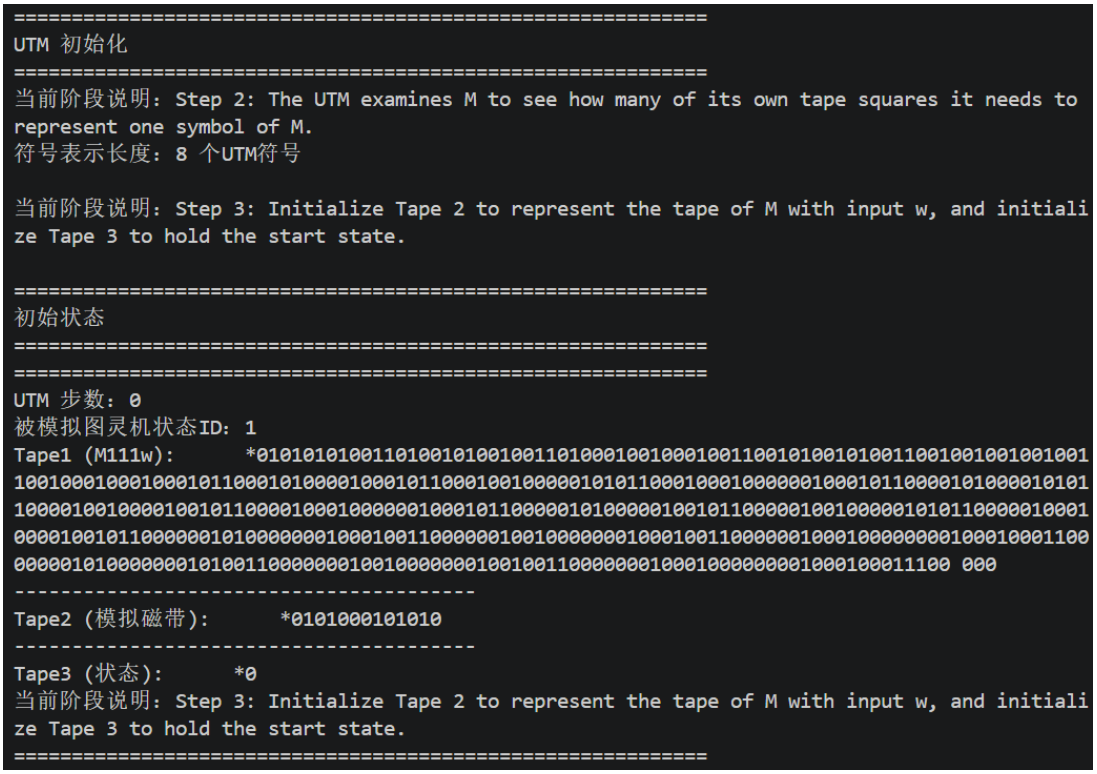
```
$ python main.py utm --sample add_tm.json --mode auto
```

UTM 初始化阶段输出：

```
==============================================================
UTM 初始化
==============================================================
当前阶段说明: Step 2: The UTM examines M to see how many
of its own tape squares it needs to represent one symbol of M.
```

```
符号表示长度：8 个UTM符号

当前阶段说明：Step 3: Initialize Tape 2 to represent the
tape of M with input w, and initialize Tape 3 to hold the
start state.
```



```
============================================================
UTM 初始化
============================================================
当前阶段说明：Step 2: The UTM examines M to see how many of its own tape squares it needs to
represent one symbol of M.
符号表示长度：8 个UTM符号

当前阶段说明：Step 3: Initialize Tape 2 to represent the tape of M with input w, and initiali
ze Tape 3 to hold the start state.

============================================================
初始状态
============================================================
============================================================
UTM 步数：0
被模拟图灵机状态ID：1
Tape1 (M111w):      *01010101001101001010010011010001001000100110010100101001100100100100100
1001000100010001011000101000010001011000100100000101011000100010000001000101100001010000101010
1000010010000100101100001000100000010001011000001010000010010110000010010000010101100000010001
0000100101100000010100000001000100110000001001000000010001001100000010001000000001000100011000
000001010000000010100110000000100100000001001001100000000100010000000001000100011100 000
----------------------------------------
Tape2 (模拟磁带):      *0101000101010
----------------------------------------
Tape3 (状态):      *0
当前阶段说明：Step 3: Initialize Tape 2 to represent the tape of M with input w, and initiali
ze Tape 3 to hold the start state.
============================================================
```

图3 UTM初始化阶段

运行过程展示三条磁带的状态变化：

```
UTM 步数：0
被模拟图灵机状态ID：1
Tape1 (M111w):      *01010101...11100 000
Tape2 (模拟磁带):      *0101000101010
Tape3 (状态):      *0

UTM 步数：6
被模拟图灵机状态ID：8
Tape1 (M111w):      *01010101...11100 000
Tape2 (模拟磁带):      000*1   101010
Tape3 (状态):      *00000000
当前阶段说明：M accepts, the UTM also accepts.

结果：接受 (ACCEPTED)
总步数：6
```

```
Tape3 (状态):      *000
当前阶段说明: If found, change the symbol and move the head marker on Tape 2 and change the S
tate on Tape 3.
============================================================

按 Enter 键继续下一步...
============================================================
UTM 步数: 5
被模拟图灵机状态ID: 6
Tape1 (M111w):      *0101010100110100101001001101000100100010011001010010100110010010010010
1001000100010001011000101000010001011000100100000101011000100010000001000101100001010000101010
1000010010000100101100001000100000010001011000001010000010010110000010010000010101100000100010
0000100101100000001010000000100010011000000100100000001000100110000001000100000000100010001100
0000010100000000101001100000000100100000001001001100000001000100000000100010001100 000
----------------------------------------
Tape2 (模拟磁带):      *0001   101010
----------------------------------------
Tape3 (状态):      *000000
当前阶段说明: If found, change the symbol and move the head marker on Tape 2 and change the S
tate on Tape 3.
============================================================

按 Enter 键继续下一步...
============================================================
UTM 步数: 6
被模拟图灵机状态ID: 8
Tape1 (M111w):      *0101010100110100101001001101000100100010011001010010100110010010010010
1001000100010001011000101000010001011000100100000101011000100010000001000101100001010000101010
1000010010000100101100001000100000010001011000001010000010010110000010010000010101100000100010
0000100101100000001010000000100010011000000100100000001000100110000001000100000000100010001100
0000010100000000101001100000000100100000001001001100000001000100000000100010001100 000
----------------------------------------
Tape2 (模拟磁带):      000*1   101010
----------------------------------------
Tape3 (状态):      *00000000
当前阶段说明: M accepts, the UTM also accepts.
============================================================


============================================================
运行结束
============================================================
当前阶段说明: M accepts, the UTM also accepts.
结果: 接受 (ACCEPTED)
总步数: 6
============================================================
```

图4 UTM运行过程（部分截图）

UTM 成功在 6 步后接受，验证了 ADD 图灵机对输入 2+3 的正确计算。

五、 总结

本项目成功实现了图灵机编码、多带图灵机和通用图灵机三个模块。编码模块采用确定性排序保证编码唯一性；多带图灵机模块支持任意数量磁带和两种运行模式；UTM 模块按照标准定义实现三带架构，能够模拟任意单带图灵机。所有模块通过 ADD 图灵机测试验证了正确性。